CODE EXPLANATION

```python
def predict(ratings, similarity, type='user'):
    if type == 'user':
        mean_user_rating = ratings.mean(axis=1)
        #We use np.newaxis so that mean_user_rating has same format as ratings
        ratings_diff = (ratings - mean_user_rating[:, np.newaxis])
        pred = mean_user_rating[:, np.newaxis] + similarity.dot(ratings_diff) / np.array([np.abs(similarity).sum(axis=1)]).T
    elif type == 'item':
        pred = ratings.dot(similarity) / np.array([np.abs(similarity).sum(axis=1)])
    return pred
```

def predict(ratings, similarity, type='user'):

- This is function with 3 inputs like ratings , similarity, and type=’user’
- Because type='user' is a default argument, if we do not provide a third input when calling the function, Python automatically runs the user-based code block.

if type == 'user':

- If the type is user the code will run , otherwise the code “elif type == 'item':” will run

mean_user_rating = ratings.mean(axis=1)

- This calculates the mathematical average rating for each individual user by calculating row-wise across the matrix.

ratings_diff = (ratings - mean_user_rating[:, np.newaxis])

- np.newaxis changes the flat list of averages into a standing vertical column matrix.
- This line subtracts each user's personal average from their actual ratings. This normalizes the data into "deviations" (showing if a user liked a movie more or less than their own usual standard).

pred = mean_user_rating[:, np.newaxis] + similarity.dot(ratings_diff) / np.array([np.abs(similarity).sum(axis=1)]).T

- this code calculates the final predicted 1-to-5 star ratings for every unwatched movie across all users
- mean_user_rating[:, np.newaxis]: Reshapes the user averages into a vertical column so they can be added back at the very end
- similarity.dot(ratings_diff): Predicts unwatched movie scores by calculating a weighted vote. It multiplies how similar users are to each other by those users' normalized movie opinions.
- / np.array([np.abs(similarity).sum(axis=1)]).T: It normalizes the raw prediction scores by dividing them by each user's total similarity weight, keeping the final ratings accurately within the 1-to-5 star scale

elif type == 'item':

- This block runs only when we explicitly pass type='item' into the function

pred = ratings.dot(similarity) / np.array([np.abs(similarity).sum(axis=1)])

- ratings.dot(similarity): Instead of looking at similar people, this predicts scores based on similar movies. It multiplies the ratings a user has given by how similar those watched movies are to the unwatched target movie.
- “np.array([np.abs(similarity).sum(axis=1)])” the raw prediction scores by dividing them by each movie's total similarity weight (summed along columns via axis=1 on the item matrix), ensuring the item-based predictions also stay cleanly within the 1-to-5 star scale.